Ya sabes moverte, gestionar archivos, permisos, procesos y pipes. Eso te hace funcional. Lo que viene ahora es lo que te hace RÁPIDO — y la velocidad es lo único que un técnico senior nota en 30 segundos viéndote teclear.
Regla de uso de esta chuleta: coge UN bloque, practícalo 15 min en una VM del laboratorio y no pases al siguiente hasta que te salga sin mirar. Instalar 20 herramientas en una tarde no sirve de nada; dominar 6 lo cambia todo.

1. Edición de línea (readline): deja de usar las flechas

Bash usa la librería **readline**: la línea que escribes es un editor de texto completo. Mover el cursor con ← ← ← es el sello del principiante. Estos atajos funcionan en bash, en psql, en python, en cualquier prompt que use readline.

Atajo	Qué hace
`Alt+B / Alt+F`	Salta una PALABRA atrás / adelante (en vez de letra a letra)
`Ctrl+W`	Borra la palabra anterior al cursor (la que estás tecleando mal)
`Alt+D`	Borra la palabra que hay delante del cursor
`Ctrl+K`	Corta desde el cursor hasta el FINAL de la línea
`Ctrl+U`	Corta desde el cursor hasta el INICIO de la línea
`Ctrl+Y`	Pega (yank) lo último que cortaste con Ctrl+W/K/U
`Alt+.`	Inserta el ÚLTIMO argumento del comando anterior (repite para ir atrás)
`Ctrl+_`	Deshacer (undo) dentro de la línea actual
`Ctrl+T`	Intercambia los dos caracteres alrededor del cursor (typos)
`Alt+T`	Intercambia las dos palabras alrededor del cursor
`Ctrl+X Ctrl+E`	Abre el comando actual en tu $EDITOR (nano/vim) y lo ejecuta al salir
`Ctrl+D`	Con línea vacía: cierra la sesión. Con texto: borra el carácter del cursor

TRUCO Nº9 — el combo del pegado:
`Ctrl+W`, `Ctrl+K` y `Ctrl+U` no borran: CORTAN a un portapapeles interno (kill ring). Escribes medio comando, te das cuenta de que falta un `mkdir` antes: `Ctrl+U` (guardas la línea) → escribes el mkdir → Enter → `Ctrl+Y` y recuperas todo. Cero retecleo.

TRUCO Nº10 — el argumento que ya escribiste:
`mkdir -p /ruta/muy/larga/logs` y ahora quieres entrar ahí: escribe `cd ` y pulsa `Alt+.` → te pega `/ruta/muy/larga/logs`. Equivalente escrito: `cd !$`. Este es, de largo, el atajo que más tiempo te va a ahorrar al día.

CUIDADO — la terminal 'colgada':
Si pulsas `Ctrl+S` la terminal se congela (control de flujo XOFF) y parece que se ha muerto: no está muerta, está en pausa. Se descongela con `Ctrl+Q`. Libera esa tecla con `stty -ixon` en tu `.bashrc` y así `Ctrl+S` te sirve para buscar hacia delante en el historial.

2. Historial de verdad: expansiones y Ctrl+R a fondo

Expansiones de historial (empiezan por `!`). Se expanden ANTES de ejecutar, así que ojo con lo que disparas:

Expansión	Qué hace
`!!`	Repite el comando anterior entero. Uso real: `sudo !!`
`!$`	Último argumento del comando anterior (versión escrita de Alt+.)
`!^`	Primer argumento del comando anterior
`!*`	TODOS los argumentos del comando anterior
`!:2`	El argumento nº 2 del comando anterior
`!apt`	Repite el último comando que empezaba por `apt`
`!?error?`	Repite el último comando que CONTENÍA 'error'
`!-3`	El comando de hace 3 posiciones
`!42`	Ejecuta el comando nº 42 que ves en `history`
`^ip^ss`	Repite el anterior cambiando la 1ª aparición de 'ip' por 'ss'
`!!:gs/dev/pro/`	Repite el anterior cambiando TODAS las 'dev' por 'pro'
`!$:h`	Último argumento, pero solo la ruta sin el fichero (h = head)

CUIDADO — `!!` es una escopeta cargada:
Se expande y ejecuta de golpe; si te equivocas de comando anterior, lo lanzas con `sudo` sin verlo. Blindaje obligatorio: añade `shopt -s histverify` a tu `.bashrc`. A partir de ahí `!!` y `!apt` te dejan el comando ESCRITO en el prompt para que pulses Enter tú. Un segundo de más, cero sustos.

Ctrl+R a fondo (búsqueda inversa incremental):

Tecla	Qué hace
`Ctrl+R`	Abre la búsqueda. Escribe trozos sueltos del comando, no el principio
`Ctrl+R (otra vez)`	Salta a la siguiente coincidencia más antigua
`Ctrl+S`	Vuelve hacia delante (requiere `stty -ixon`)
`← → o Alt+E`	Sale de la búsqueda y deja el comando EDITABLE sin ejecutarlo
`Enter`	Lo ejecuta tal cual
`Ctrl+G`	Cancela y te devuelve la línea vacía, sin ejecutar nada
`Ctrl+O`	Ejecuta y carga el SIGUIENTE comando del historial (repetir secuencias)
`history | grep x`	Búsqueda 'a lo bruto' cuando quieres ver todas las coincidencias

Configura el historial como un profesional (al final de `~/.bashrc`):

```
HISTSIZE=100000            # comandos en memoria
HISTFILESIZE=200000        # comandos en disco
HISTCONTROL=ignoreboth:erasedups
HISTTIMEFORMAT='%F %T '    # fecha y hora en history
HISTIGNORE='ls:ll:cd:pwd:clear:history:exit'
shopt -s histappend histverify cmdhist
PROMPT_COMMAND='history -a'  # graba al instante
```

`ignoreboth` = no guarda duplicados seguidos ni líneas que empiezan por espacio. `histappend` + `history -a` evitan que dos terminales abiertas se pisen el historial al cerrarse — el clásico 'he perdido los comandos de hoy'.

CUIDADO — tu historial es un fichero de texto plano:
`mysql -u root -pMiClave123` queda escrito para siempre en `~/.bash_history` y además visible en `ps aux` mientras corre. Dos hábitos: (1) pon un ESPACIO delante del comando y con `ignorespace` no se guarda; (2) si ya lo has escrito, `history -d NÚMERO` borra esa línea y `history -w` guarda el cambio. En entrevista, mencionar esto te sube el nivel percibido de golpe.

3. Expansión de llaves y globs: escribir menos, abarcar más

Patrón	Qué hace
`cp cfg{,.bak}`	Copia de seguridad instantánea: se expande a `cp cfg cfg.bak`
`mv log{,.old}`	Mismo truco para renombrar sin repetir el nombre
`mkdir -p web/{css,js,img}`	Crea las tres carpetas de una tacada
`touch nota{1..5}.txt`	Rangos: nota1.txt ... nota5.txt. También `{a..f}`, `{00..10}`
`shopt -s globstar`	Activa `**`: `ls **/*.log` busca en TODAS las subcarpetas
`shopt -s extglob`	Activa `!(...)`: `rm !(*.conf)` borra todo MENOS los .conf
`ls ?.txt`	`?` = un carácter cualquiera; `[0-9]` = rango de caracteres

TRUCO Nº11 — comprueba antes de destruir:
Cualquier expansión o glob la puedes auditar poniendo `echo` delante: `echo rm -rf viejo/*` te enseña EXACTAMENTE qué se va a ejecutar sin ejecutar nada. Con `rm -rf` esto no es paranoia, es procedimiento.

4. Herramientas modernas que sí usan los profesionales

Todas son binarios sueltos escritos en Rust/Go: rápidos y sin dependencias. En Ubuntu 24.04 LTS y 26.04 LTS casi todas están ya en los repos oficiales (`apt`), así que no necesitas compilar nada. Instálalas en este orden.

Herramienta	Qué hace, cómo se instala y por qué merece la pena
`fzf`	**Buscador difuso universal.** `sudo apt install fzf` → activa con `eval "$(fzf --bash)"` en el .bashrc. Te reescribe `Ctrl+R` (historial), `Ctrl+T` (ficheros al cursor) y `Alt+C` (cd difuso). Es la herramienta con más retorno por minuto invertido.
`ripgrep`	**grep 10-100× más rápido.** `sudo apt install ripgrep` → binario `rg`. Recursivo por defecto, respeta `.gitignore`, colorea y numera: `rg -n "error" /var/log`. `rg -i` ignora mayúsculas, `rg -C3` da contexto.
`fd`	**find sin sintaxis críptica.** `sudo apt install fd-find` → OJO: el binario se llama `fdfind`, hazte `alias fd=fdfind`. `fd conf /etc` vs el clásico `find /etc -name "*conf*"`.
`bat`	**cat con sintaxis y números de línea.** `sudo apt install bat` → binario `batcat` (`alias cat='batcat -pp'`). Ideal para leer configs y scripts.
`eza`	**ls moderno.** `sudo apt install eza`. `eza -la --git --group-directories-first` y `eza --tree --level=2` sustituyen a `ls -al` y a `tree`.
`zoxide`	**cd que aprende.** `sudo apt install zoxide` + `eval "$(zoxide init bash)"`. Tras visitar una carpeta, `z proy` te lleva a `/home/tu/proyectos/proyecto-x` desde cualquier sitio. `zi` abre el selector con fzf.
`tldr`	**man en 10 líneas de ejemplos.** `sudo apt install tldr` (o `tealdeer`) → `tldr tar` te da los 5 usos reales en vez de 400 líneas de manual. `tldr --update` refresca la caché.
`jq`	**Procesador de JSON.** `sudo apt install jq`. Obligatorio en cuanto toques APIs, AWS CLI o Docker: `docker inspect X | jq '.[0].State'`.
`btop`	**top del siglo XXI.** `sudo apt install btop`. Gráficas de CPU/RAM/red y matar procesos con el ratón o con `f` para filtrar.
`ncdu / duf`	**Quién se come el disco.** `sudo apt install ncdu duf`. `ncdu /var` navega el árbol por tamaño; `duf` es un `df -h` legible. El día que un servidor se quede sin espacio, esto te salva.
`git-delta`	**Diffs de Git legibles.** `sudo apt install git-delta`. Se nota mucho cuando llegues al bloque de Git.
`atuin`	**Historial en SQLite con buscador.** No está en los repos: `bash <(curl -sSf https://setup.atuin.sh)` + `eval "$(atuin init bash)"`. Guarda directorio, código de salida y duración de cada comando. Opcional — primero exprime el `Ctrl+R` nativo.

TRUCO Nº12 — fzf es más que Ctrl+R:
Con la integración activa, escribe `**` y pulsa Tab: `vim **<Tab>` te abre un selector de ficheros, `cd **<Tab>` de carpetas, `ssh **<Tab>` de hosts conocidos y `kill -9 **<Tab>` te lista procesos para elegir el PID sin buscarlo. Combínalo con vista previa: `export FZF_CTRL_T_OPTS="--preview 'batcat -n --color=always {}'"`.

CUIDADO — nombres de binario en Debian/Ubuntu:
`bat` se instala como `batcat` y `fd` como `fdfind` (conflicto de nombres con otros paquetes). Si copias comandos de un tutorial y te dice 'command not found', no es que esté mal instalado: te falta el alias. Compruébalo siempre con `type -a bat` o `apt-cache policy fzf` para ver qué versión tienes de verdad.

CUIDADO — no te enamores de las herramientas:
En un servidor de producción o en un contenedor solo habrá `ls`, `grep`, `find` y `vi`. Usa las modernas para ir rápido en TU máquina, pero mantén los clásicos en los dedos: en una entrevista técnica te pedirán `grep -r` y `find -name`, no `rg`. Nunca metas `rg` o `eza` dentro de un script que tenga que correr en otra máquina.

5. Tu .bashrc: alias, funciones y prompt

Todo lo que sigue va en `~/.bashrc` (o mejor: en `~/.bash_aliases`, que Ubuntu ya carga solo). Recarga con `source ~/.bashrc` — no hace falta cerrar sesión.

Alias que ganan tiempo de verdad

```
alias ll='eza -lah --git --group-directories-first'
alias ..='cd ..'   ; alias ...='cd ../..'
alias grep='grep --color=auto'
alias df='df -hT' ; alias du='du -h'
alias ports='ss -tulpn'
alias myip='curl -s ifconfig.me'
alias please='sudo $(history -p !!)'
alias sudo='sudo '   # <- el espacio final es clave
```

TRUCO Nº13 — el espacio de `alias sudo='sudo '`:
Por defecto bash NO expande alias después de `sudo`, así que `sudo ll` falla. Terminar el alias en un espacio le dice a bash que mire también la palabra siguiente. Detalle pequeño, lo conoce muy poca gente.

Funciones: lo que un alias no puede hacer (aceptan argumentos)

```
mkcd() { mkdir -p "$1" && cd "$1"; }

bak() { cp -a "$1" "$1.$(date +%F_%H%M).bak"; }

extract() {
  case "$1" in
    *.tar.gz|*.tgz) tar xzf "$1" ;;
    *.tar.bz2)      tar xjf "$1" ;;
    *.zip)          unzip "$1"   ;;
    *.gz)           gunzip "$1"  ;;
    *) echo "Formato no soportado: $1" ;;
  esac
}

up() { local d=; for i in $(seq 1 ${1:-1});
       do d="../$d"; done; cd "$d"; }   # up 3
```

Fíjate en las **comillas dobles alrededor de** `"$1"`: sin ellas, un fichero con espacios en el nombre rompe la función. Es el error nº1 al escribir bash y el primero que mira un revisor.

PS1: que el prompt te chive el estado

Código	Qué muestra
`\u \h`	usuario y hostname (saber si estás en local o en el servidor)
`\w / \W`	ruta completa / solo la carpeta actual
`\$`	muestra `#` si eres root y `$` si no — tu primer aviso de peligro
`\t`	hora (para saber cuánto tardó lo anterior)
`$?`	código de salida del último comando: 0 = bien, distinto de 0 = falló

```
PS1='\[\e[32m\]\u@\h\[\e[0m\]:'
PS1+='\[\e[34m\]\w\[\e[0m\]'
PS1+='$(s=$?; [ $s -ne 0 ] && echo " \[\e[31m\][$s]\[\e[0m\]"
)'
PS1+='\$ '
PROMPT_DIRTRIM=3   # acorta rutas larguísimas
```

CUIDADO — los corchetes `\[ \]` no son decorativos:
Encierran los códigos de color para que bash no los cuente como caracteres visibles. Si los olvidas, el prompt 'funciona'... hasta que editas una línea larga o usas `Ctrl+R` y el texto empieza a sobrescribirse encima. Cuando veas ese comportamiento en la terminal de alguien, ya sabes por qué es.

TRUCO Nº14 — atajo si no quieres pelearte con PS1:
`sudo apt install starship` (Ubuntu 25.04+) y añade `eval "$(starship init bash)"`. Te da rama de Git, estado del repo, versión de Python/Node y código de error automáticamente. Eso sí: aprende primero qué es `\u@\h:\w`, porque en un servidor ajeno solo tendrás el prompt pelado.

6. tmux: varias sesiones sin morir en el intento

El caso real: estás actualizando un servidor por SSH, se te cae el wifi y el proceso muere a medias dejando el sistema roto. Con tmux el proceso vive en el servidor, no en tu conexión: reconectas y sigues donde estabas. Por eso se pregunta en entrevistas.

Modelo mental: sesión (el trabajo entero) → ventanas (pestañas) → paneles (divisiones de pantalla). Todo se maneja con un **prefijo**: `Ctrl+B` y luego la tecla.

Comando / atajo	Qué hace
`tmux new -s lab`	Crea una sesión llamada 'lab'
`Ctrl+B d`	Detach: sales dejándolo TODO corriendo
`tmux ls`	Lista las sesiones vivas
`tmux a -t lab`	Vuelve a entrar en la sesión 'lab'
`Ctrl+B c`	Nueva ventana · `Ctrl+B n/p` siguiente/anterior · `Ctrl+B 0-9`
`Ctrl+B ,`	Renombra la ventana (ponles nombre: logs, db, deploy)
`Ctrl+B %`	Divide en vertical · `Ctrl+B "` divide en horizontal

Comando / atajo	Qué hace
`Ctrl+B ←↑↓→`	Salta entre paneles · `Ctrl+B z` amplía/restaura el panel actual
`Ctrl+B x`	Cierra el panel · `Ctrl+B &` cierra la ventana
`Ctrl+B [`	Modo copia: scroll y búsqueda con `/`. Sales con `q`
`Ctrl+B *`	Panel flotante (novedad de tmux 3.7, junio 2026)
`Ctrl+B ?`	Lista TODOS los atajos. Tu chuleta dentro de tmux

`~/.tmux.conf` **mínimo** (recarga con `tmux source-file ~/.tmux.conf`):

```
set -g mouse on            # ratón para paneles y scroll
set -g history-limit 100000
set -g base-index 1        # ventanas desde 1, no 0
setw -g mode-keys vi
bind r source-file ~/.tmux.conf \; display 'OK'
```

TRUCO Nº15 — el alias que usarás cada día:
`alias tm='tmux attach 2>/dev/null || tmux new -s main'` → un solo comando que se engancha a tu sesión si existe y la crea si no. Monta tu laboratorio así: ventana 1 el servidor, ventana 2 los logs con `tail -f`, ventana 3 libre. Cierras el portátil, vuelves mañana, todo sigue igual.

CUIDADO — tmux vive en la máquina donde lo lanzas:
Si abres tmux en tu portátil y desde ahí haces SSH, la caída de red te sigue matando el proceso. El orden correcto es: SSH primero, tmux DESPUÉS, dentro del servidor. Es el fallo típico del primer día de prácticas.

7. Combos de guerra: lo que se usa el primer día

Encadenar y automatizar

Combo	Para qué
`sudo !!`	Relanza como root el comando que acaba de dar 'Permission denied'
`cmd1 && cmd2`	Ejecuta el 2º SOLO si el 1º va bien · `||` solo si FALLA
`cmd &> out.log`	Salida y errores al mismo fichero · `2>/dev/null` calla errores
`echo x | sudo tee -a /etc/f`	La forma CORRECTA de escribir en ficheros de root
`find . -name '*.log' -print0 | xargs -0 rm`	Borrado masivo seguro con nombres raros
`ls *.txt | xargs -I{} mv {} {}.old`	`{}` = cada elemento, uno a uno
`cat ips.txt | xargs -P8 -I{} ping -c1 {}`	`-P8` = 8 en paralelo. Brutal en listas largas
`watch -n2 'ss -s'`	Repite un comando cada 2 s y refresca en pantalla
`timeout 10 comando`	Lo mata si tarda más de 10 s
`comando | tee salida.txt`	Ves la salida Y la guardas a la vez

CUIDADO — `sudo echo "x" >> /etc/hosts` NO funciona:
El `sudo` afecta al `echo`, pero la redirección `>>` la hace TU shell, que no es root → 'Permission denied'. La solución profesional es `echo "x" | sudo tee -a /etc/hosts`. Pregunta clásica de entrevista junior y motivo de mil minutos perdidos.

Diagnóstico rápido de una máquina

Comando	Qué te dice
`ss -tulpn`	Qué puertos están abiertos y QUÉ proceso los ocupa (`netstat` está obsoleto)
`ip -br a`	IPs de todas las interfaces en modo resumen legible
`systemctl --failed`	Servicios que han petado. Primer comando al entrar a un servidor
`systemctl status nginx`	Estado + últimas líneas de log de un servicio
`journalctl -u nginx -f`	Log EN VIVO de ese servicio · `--since '10 min ago'`
`journalctl -p err -b`	Solo errores desde el último arranque
`du -sh * | sort -h`	Ordena carpetas por tamaño (`-h` de sort entiende K/M/G)
`df -hT`	Espacio y tipo de sistema de ficheros de cada partición

Comando	Qué te dice
`free -h · uptime`	RAM disponible y carga media del sistema
`lsof -i :8080`	Quién está usando el puerto 8080
`ps aux --sort=-%mem | head`	Los procesos que más RAM comen, ordenados

SSH y copias: lo que separa a un junior de un técnico

Configura `~/.ssh/config` una vez y olvídate de IPs, puertos y usuarios:

```
Host lab
    HostName 192.168.1.50
    User asir
    Port 22
    IdentityFile ~/.ssh/id_ed25519

# Ahora basta con:  ssh lab   ·   scp fich lab:/tmp/
```

Comando	Qué hace
`ssh-keygen -t ed25519`	Genera tu par de claves moderno (no uses RSA de 2048 ya)
`ssh-copy-id lab`	Copia tu clave pública al servidor: se acabaron las contraseñas
`rsync -avh --progress o/ d/`	Copia solo lo que cambió, con progreso. Mejor que `scp`
`rsync -avhn o/ d/`	`-n` = simulacro (dry-run): enseña qué haría SIN tocar nada

CUIDADO — la barra final en rsync cambia el resultado:
`rsync -av origen/ destino/` copia el CONTENIDO de origen dentro de destino. `rsync -av origen destino/` copia la CARPETA origen dentro de destino. Un carácter de diferencia y un backup mal montado. Por eso se lanza siempre antes con `-n`.

Filtrado de texto que impresiona

Herramienta	Uso típico
`awk '{print $1, $9}'`	Imprime columnas concretas de cada línea
`cut -d: -f1 /etc/passwd`	Corta por delimitador: lista de usuarios del sistema
`sort | uniq -c | sort -rn`	El ranking: cuenta repeticiones y ordena de mayor a menor
`sed -i.bak 's/dev/pro/g' f`	Sustituye en el fichero y deja copia `.bak` de seguridad
`tr -s ' '`	Aplasta espacios múltiples en uno (antes de cortar por columnas)
`wc -l · head -n5 · tail -f`	Contar, primeras líneas, seguir un log en vivo

TRUCO Nº16 — el one-liner que te contratan por saber:
`awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head` → las 10 IPs que más peticiones han hecho. Mismo patrón para logs de SSH, de correo o de firewall: **extraer columna → ordenar → contar → ordenar por número**. Apréndete ESE patrón, no el comando.

8. Hábitos que delatan (para bien o para mal)

Señal	Lo que se espera de ti
`cat fich | grep x`	Sobra el `cat`: es `grep x fich`. Lo detectan al vuelo
`ls | grep .txt`	Se hace con globs: `ls *.txt`. Parsear `ls` es mala práctica
`Usar which`	Mejor `command -v` (portable) o `type -a` (te dice si es alias)
`Variables sin comillas`	`rm $f` con espacios borra cosas que no querías. Usa `"$f"`
`chmod 777`	Nunca. Piensa qué necesita de verdad: 644 ficheros, 755 ejecutables
`kill -9 de primeras`	Prueba `kill PID` (TERM) y deja que el proceso cierre limpio
`Trabajar como root`	`sudo` puntual y trazado en `/var/log/auth.log`. Nunca `sudo -i` por comodidad

TRUCO Nº17 — antes de ejecutar algo que no entiendes:
`type -a comando` (¿es alias, función o binario?), `tldr comando` (ejemplos reales), `man comando` + `/` para buscar dentro, y `comando --help | grep -i palabra`. En una prueba técnica, decir en voz alta 'primero compruebo qué hace' suma más puntos que acertar por casualidad.

TRUCO Nº18 — dónde practicar esto:
En una VM Ubuntu de tu laboratorio, no en el host. Haz snapshot antes de trastear con `.bashrc`, permisos o `rm -rf`: romper la máquina y restaurarla en 20 segundos es parte del aprendizaje. Un `.bashrc` mal escrito puede dejarte sin prompt utilizable, y es mejor que eso pase en un entorno desechable.

9. Patrones que te ahorran memorizar

Flags que significan lo mismo en casi todo

Flag	Significado habitual
`-r / -R`	Recursivo: aplícalo a la carpeta y a todo lo de dentro (`cp`, `chmod`, `rg`)
`-a`	All / archive: todo, incluidos ocultos y metadatos (`ls -a`, `cp -a`, `rsync -a`)
`-v / -q`	Verbose (cuéntame qué haces) / quiet (cállate, útil en scripts)
`-n`	Dry-run o 'no': simula sin tocar nada (`rsync -n`, `mv -n`, `sed -n`)
`-i`	Interactivo: pregunta antes de cada acción destructiva (`rm -i`, `cp -i`)
`-f`	Force: no preguntes, no avises. El flag que más servidores ha roto
`-h`	Human-readable en unos (`df -h`), help en otros. Ante la duda, `--help`
`-p`	Parents (`mkdir -p`) o preserve (`cp -p`): crea/conserva la estructura
`-y`	Responde sí a todo (`apt install -y`), imprescindible al automatizar

Cuando algo 'no funciona': orden de comprobación

Paso	Comando y qué descartas
`1. ¿Existe?`	`type -a cmd` — ¿es alias, función, binario o directamente no está?
`2. ¿Permisos?`	`ls -l fichero` + `id` — ¿eres el dueño?, ¿está el bit x?
`3. ¿Ruta?`	`echo $PATH`, `pwd` — el 90% de los 'no such file' son rutas relativas
`4. ¿Arrancado?`	`systemctl status servicio` — ¿está activo o murió en el arranque?
`5. ¿Escucha?`	`ss -tulpn | grep puerto` + `sudo ufw status` — proceso y firewall
`6. ¿Sitio?`	`df -h` y `df -i` — disco lleno o inodos agotados, el fallo fantasma
`7. ¿Qué dice?`	`journalctl -xe` / `tail -50 /var/log/syslog` — el log SIEMPRE lo cuenta

Ese orden es literalmente el guion de un técnico de soporte: se va de lo barato y local a lo caro y remoto. Recítalo en voz alta en una prueba técnica y transmites método, que es lo que se evalúa cuando aún no tienes experiencia.

10. Checklist de prioridades (máximo impacto / mínimo tiempo)

Cuándo	Qué dominar
`Hoy · 20 min`	`Alt+.`, `Ctrl+W/U/Y`, `Ctrl+R` editando con →, `sudo !!` y `!$`. Sin instalar NADA. Es el 40% de la ganancia total
`Día 2 · 30 min`	Bloque de historial en el `.bashrc` (HISTSIZE, ignoreboth, histverify, histappend) + `stty -ixon`
`Día 3 · 30 min`	Instala SOLO `fzf` y `ripgrep`. Fuerza su uso una semana antes de añadir nada más
`Semana 1`	Tus 10 alias + 3 funciones (`mkcd`, `bak`, `extract`) + PS1 con código de error. Guarda el `.bashrc` en un USB o repo
`Semana 2`	`tmux`: sesión, detach, attach, ventanas y paneles. Trabaja SIEMPRE dentro de tmux hasta que sea automático
`Semana 2-3`	`eza`, `bat`, `zoxide`, `tldr`, `jq` + aliases. Ya con el resto asentado
`Semana 3`	Bloque de diagnóstico: `ss -tulpn`, `journalctl -u -f`, `systemctl --failed`, `du -sh * | sort -h`. Es lo que te van a pedir en prácticas
`Semana 4`	`~/.ssh/config` + `ssh-copy-id` + `rsync -avhn`. Practícalo contra tus VMs
`Siguiente hito`	Con esto encadenas directo a Bash scripting: las funciones del `.bashrc`, las comillas, `$?` y `&&` YA son scripting. Luego Git (+ `git-delta`), Docker (`jq` + `lazydocker`) y AWS CLI (`jq` otra vez)

CUIDADO — lo que NO debes hacer todavía:
No te instales zsh + oh-my-zsh + 30 plugins ni un Neovim configurado desde cero. Es tiempo de configuración disfrazado de aprendizaje, y el día que entres por SSH a un servidor ajeno solo tendrás bash pelado. Domina bash puro primero; lo demás es cosmética que se aprende en una tarde cuando ya sabes lo de debajo.

Regla de oro nivel 3: la velocidad no viene de saber más comandos, viene de no repetir trabajo — no reescribir lo que ya está en el historial, no buscar lo que fzf encuentra, no reconectar lo que tmux mantiene vivo. Y antes de pulsar Enter en algo destructivo, dilo en voz alta: si no sabes explicar qué va a pasar, todavía no toca ejecutarlo.